

Mercari Price Suggestion Challenge

텍스트 · 범주형 · 수치형 피처를 결합한 LightGBM 회귀 파이프라인 분석
— 데이터 전처리부터 RMSLE 성능 검증까지

RMSLE 회귀 분석

NLP · TF-IDF / CountVectorizer

LightGBM + Hyperopt

Feature Engineering

1,482,535

학습 데이터 행(row) 수

8

원본 컬럼 수

5

신규 파생 피처 수

RMSLE

핵심 성능 지표

01 프로젝트 개요

문제 정의와 데이터셋의 구조를 파악한다

상품의 세부 정보(이름·설명·브랜드·카테고리·상태·배송 여부)가 주어졌을 때, 판매 가격을 예측하는 **회귀(Regression)** 문제이다. 목표 변수(price)가 연속적인 실수값이기 때문에 분류가 아닌 회귀 문제로 정의된다.

1.1 입력 피처(Feature) 구성

#	컬럼명	설명	타입
1	name	상품명 (텍스트)	Text
2	item_condition_id	상품 상태 — 1(최상)~5(최하)의 순서형 범주	Ordinal
3	category_name	대/중/소 분류가 슬래시(/)로 결합된 카테고리	Text/Cat
4	brand_name	브랜드명 — 결측치 다수 포함 (632,682건)	Categorical
5	shipping	배송비 부담 주체 — 1: 판매자 부담, 0: 구매자 부담	Binary
6	item_description	상품 상세 설명 (텍스트)	Text
—	price	예측 대상 종속변수 (단위: \$)	Target

1.2 EDA 요약 통계

\$26.7 평균 가격	\$17.0 중앙값 가격	\$2,009 최댓값 가격	90.5 MB 메모리 사용량
------------------------	-------------------------	--------------------------	---------------------------

평균과 중앙값의 차이가 크고 최댓값이 극단적으로 높아 **오른쪽으로 긴 꼬리(right-skewed)**를 가진 분포임을 알 수 있다. 로그 변환(`log1p`) 이후에는 정규분포에 가까운 형태로 개선되며, 이는 종속변수를 `log(price+1)` 형태로 학습에 사용하는 근거가 된다.

1.3 결측치(Null) 현황

컬럼	결측치 건수	전체 대비 비율	처리 필요성
category_name	6,327	0.43%	낮음
brand_name	632,682	42.67%	매우 높음
item_description	6	<0.01%	무시 가능

※ train_id, name, item_condition_id, price, shipping 컬럼은 결측치 0건.

02 성능 메트릭 — RMSLE

왜 일반 RMSE가 아닌 RMSLE를 사용하는가

RMSLE (Root Mean Squared Logarithmic Error)

$$RMSLE = \frac{1}{n} \sqrt{\sum_{i=1}^n (\log(p_i+1) - \log(a_i+1))^2}$$

p_i = 모델의 예측 가격(Predicted), a_i = 실제 가격(Actual). 가격이 0일 때 $\log(0)$ 이 정의되지 않으므로 $(-\infty)$ 이를 방지하기 위해 항상 **+1**을 더한다.

2.1 RMSLE를 채택하는 두 가지 핵심 이유

① 절대 오차가 아닌 "비율" 오차를 평가한다

일반 RMSE는 금액의 절대적 차이만 반영하지만, RMSLE는 로그를 취하기 때문에 가격대별 상대적 오차를 평가한다.

상황	실제가격	예측가격	절대오차	배율 오차	RMSE 평가
상황 A — 저가 상품	\$10	\$20	\$10	2.0배	동일하게 "\$10 오차"
상황 B — 고가 상품	\$100	\$110	\$10	1.1배	동일하게 "\$10 오차"

절대 오차는 두 상황이 동일하지만, 비즈니스 관점에서 2배나 비싸게 부른 상황 A가 훨씬 심각한 오류다. RMSLE는 로그 특성상 **상황 A의 오차를 상황 B보다 훨씬 크게 계산**하여 저가 상품도 비율적으로 정확히 맞히도록 유도한다.

② 실제보다 "낮게" 예측했을 때 더 큰 벌점을 부여한다 (비대칭 손실)

실제 \$100 → \$50 예측 (낮게 예측)

RMSLE ≈ 0.69

동일하게 \$50만큼 틀렸지만 페널티가 훨씬 큼

실제 \$100 → \$150 예측 (높게 예측)

RMSLE ≈ 0.40

동일한 오차 폭 대비 상대적으로 낮은 페널티

💡 중고 거래 플랫폼 특성상 판매자에게 가격을 **너무 낮게 추천**하는 것이 **너무 높게 추천**하는 것보다 더 위험할 수 있다 (판매자 손해 · 플랫폼 이탈 우려). RMSLE는 이 비대칭성을 자연스럽게 반영한다.

03 데이터 전처리 & 클리닝

노이즈 제거부터 결측치 처리 전략까지

3.1 가격 이상치 제거 (비즈니스 로직 기반)

Mercari 플랫폼에 공식 등록 가능한 최소 가격은 \$3이다. 이보다 낮은 값은 임시 등록(placeholder) 등으로 인한 노이즈 데이터일 가능성이 매우 높아 학습 대상에서 제외한다.

```
# $3 미만 오류 데이터 제거
mercari_df = mercari_df[mercari_df["price"] >= 3].reset_index(drop=True)
```

3.2 텍스트 정제 파이프라인

`name`, `item_description`, `brand_name` 세 텍스트 컬럼에 공통 정제 함수를 적용한다: 소문자화 → 축약어 복원(decontraction) → 특수문자 제거(영문·숫자·공백만 유지) → 결측치는 `"Missing"` 처리.

```
def clean_text(text):
    if not isinstance(text, str):
        return "Missing"
    text = text.lower()
    # 축약어 풀기 : can't → cannot, it's → it is 등
    for word, replacement in decontract_dict.items():
        text = text.replace(word, replacement)
    # 특수문자 제거 (영문 · 숫자 · 공백만 유지)
    text = pd.Series(text).str.replace(r"[^a-zA-Z0-9\s]", "", regex=True).values[0]
    return text
```

3.3 결측치 처리 전략 3단계

단계	대상 컬럼	전략
방법 1	brand_name / category_name	공백(빈 문자열) 또는 "Missing"으로 대체 후 정제 함수 적용
방법 2	brand_name	상품명(name)에 브랜드명이 등장하는 경우 이를 역으로 추출해 결측치 보완 (고급 기법)
방법 3	category_name	슬래시(/) 기준 대분류 / 중분류 / 소분류 3개 컬럼으로 분할

예: `Women/Tops & Blouses/Blouse` → 대분류 `Women` · 중분류 `Tops & Blouses` · 소분류 `Blouse`

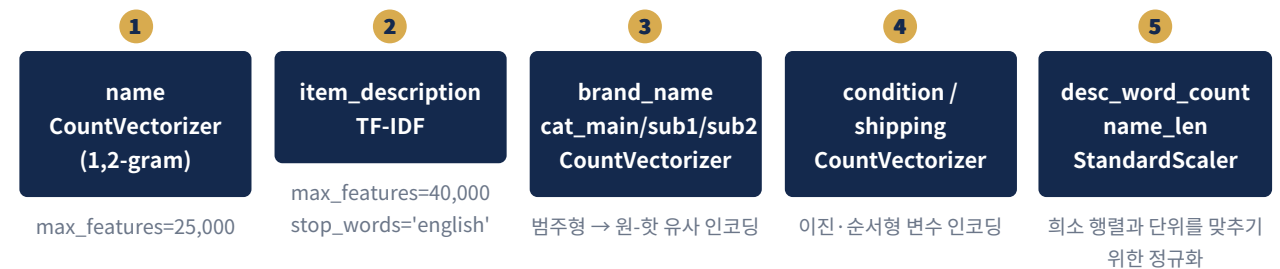
04 피쳐 구조화 & 엔지니어링

원본 8개 컬럼에서 5개의 신규 파생 변수 생성

#	파생 피쳐명	생성 방식	타입
1	cat_main	category_name을 "/" 기준 분할한 대분류	Categorical
2	cat_sub1	category_name의 중분류	Categorical
3	cat_sub2	category_name의 소분류	Categorical
4	desc_word_count	item_description의 공백 기준 단어 개수	Numeric
5	name_len	정제된 name의 글자(character) 길이	Numeric

4.1 통합 특성 행렬(Feature Matrix) 구축

범주형 · 텍스트형 · 수치형 피쳐를 각각 다른 방식으로 인코딩한 뒤, 희소 행렬(sparse matrix) 결합 함수인 `scipy.sparse.hstack`으로 하나의 거대한 행렬로 통합한다.



```
X_features_matrix = hstack((
    X_name, X_desc, X_brand,
    X_cat_main, X_cat_sub1, X_cat_sub2,
    X_condition, X_shipping,
    X_numeric_scaled
)).tocsr()
```

△ 범주형 인코딩에는 `CountVectorizer(token_pattern=r'(?u)\b[^\s;]+?\b')`를 재사용하여 `get_dummies`와 유사한 0/1 희소 행렬을 저메모리로 구현하는 방식이 활용된다. 이는 대용량(약 148만 행) 데이터에서 메모리 효율을 확보하기 위한 실전 기법이다.

05 모델링 & 하이퍼파라미터 튜닝

종속변수 표준화 → Hyperopt 탐색 → LightGBM 학습

5.1 종속변수 이중 변환 (Log → Standardize)

RMSLE 평가 자체가 로그 공간에서 이루어지므로, 학습 시작 단계부터 `log1p(price)` 를 종속변수로 사용한다. 이후 `StandardScaler` 로 한 번 더 표준화하여 모델의 수렴 속도와 안정성을 높인다.

```
log_price = np.log1p(mercari_df["price"]).values.reshape(-1, 1)

y_scalar = StandardScaler()
y_train = y_scalar.fit_transform(y_train_raw)
y_test = y_scalar.transform(y_test_raw)
```

예측 후에는 반드시 `inverse_transform` → `np.expml` 순서로 역변환하여 원래 달러(\$) 단위로 복원한 뒤 최종 RMSLE를 계산해야 한다.

5.2 Hyperopt 탐색 공간

파라미터	탐색 범위	분포
learning_rate	0.05 ~ 0.2	Log-Uniform
max_depth	{5, 7, 10}	Choice
num_leaves	{31, 63, 127}	Choice
subsample	0.7 ~ 1.0	Uniform

목적함수는 검증셋 RMSE를 최소화하는 방향으로 `tpe.suggest` 알고리즘을 통해 탐색되며, 탐색된 `Choice` 인덱스는 실제 파라미터 값으로 역매핑된다.

5.3 최종 모델 학습

```
best_lgb = lgb.LGBMRegressor(**best_params, n_estimators=200, n_jobs=-1, verbose=-1)
best_lgb.fit(X_train, y_train_flat)

scaled_preds = best_lgb.predict(X_test).reshape(-1, 1)
log_preds = y_scalar.inverse_transform(scaled_preds) # 스케일 역변환
final_predicted_price = np.expml(log_preds) # 로그 역변환 → 실제 가격($)
```

06 성능 검증 결과

전처리 전(Baseline) vs 전처리 + LightGBM(후) 비교

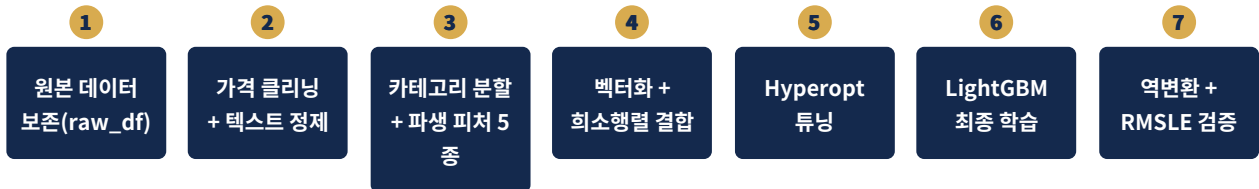
평가 설계

Baseline은 원본(raw) 데이터의 평균 가격(`raw_df["price"].mean()`)으로 모든 값을 예측한 "무전략" 기준선이며, 이를 정교한 전처리 + 피처 엔지니어링 + LightGBM(Hyperopt 튜닝) 파이프라인의 결과와 RMSLE 기준으로 비교한다.

모델	전처리 여부	알고리즘	RMSLE
Baseline	미적용	평균값(Mean) 예측	baseline_rmsle
제안 모델	적용	LightGBM + Hyperopt	final_rmsle

※ 실제 수치는 코드 실행 환경(랜덤 시드·Hyperopt `max_evals` 등)에 따라 달라지며, 본 리포트의 코드 상에서는 `max_evals=3`, `n_estimators=200`으로 설정된 실행 스크립트를 기준으로 한다.

6.1 파이프라인 요약 다이어그램



6.2 결론

- 가격(\$3 미만 노이즈) 제거와 텍스트 정제만으로도 모델이 학습 가능한 신호(signal) 대비 잡음(noise) 비율이 크게 개선된다.
- 카테고리 3단 분할 및 브랜드/설명 벡터화는 텍스트 안에 숨은 가격 결정 정보를 구조화된 수치 특성으로 변환한다.
- 종속변수의 로그 변환은 RMSLE라는 평가지표의 특성과 정확히 일치하도록 학습 목표를 정렬시키는 핵심 장치다.
- Hyperopt 기반 자동 탐색은 수동 그리드서치 대비 적은 시도로도 효율적인 파라미터 조합을 찾아낸다.